

Session Transcript

TM-022 Wave 1 · impacket · Complete Assessment Log

Project	iHBV-TM-022
Author	HoneyBadger Vanguard LLC
Date	2026-08-14
Classification	Internal — Purple Team Research

Session Transcript — TM-022 Wave 1 (impacket)

Program: TM-022 Purple Validation Harness — Wave 1 **Project code:** iHBV-TM-022 **Assessor session:** Claude Code (models: Opus 4.8, then Opus 4.7) **Date:** 14 August 2026 **Working branch:** `claude/impacket-adversarial-wave-1-39koer` **Repository:** MoSLoF/impacket **Distribution:** Internal — Purple Team Research

This transcript is a faithful reconstruction of the assessment session, presented chronologically. Full command output and JSON blobs are abridged for readability; complete raw evidence is committed under `tm022_wave1_harness/` and reproducible via the commands listed in each phase. Timing marks reflect narrative order, not wall-clock.

Phase 0 — Session start and prompt

The session opened with the TM-022 Wave 1 briefing pack pre-loaded as the initial user prompt. Key parameters:

- Mission: structured adversarial assessment of `impacket`, calibrated against a known-good edge case (EC-01), then extended through five additional spec-derived boundary conditions (EC-02 – EC-06).
- Guardrails:
 - *Spec first, code second.*
 - *Fixture-driven, not lab-dependent* (no live AD).
 - *Both versions on every case* (0.13.1 baseline and patched master).
 - *Same assumption, different file* (cross-file assumption sweep).
 - *Silent-wrong-result outranks crash* (operational risk prioritisation).
- Scope: `impacket` only (Rubeus, bloodyAD, and detection-side work are separate sessions).
- Stop condition: full coverage of EC-01 through EC-06 is worth more than partial coverage of more cases.

Phase 1 — Environment discovery and setup

Assessor: confirmed working directory state and located the code under test.

```
$ git log --oneline -3
4c09897 Fix some SMB relay server syntax error (#2245)
8ea54fe Modernize byte string normalization (#2243)
243d64a Fix IndexError in CCache.getCredential ... 3-part SPNs ... (#2242)
```

Findings:

- The working tree already contains the fix commit (243d64a , PR #2242).
- The pre-fix baseline can be reached as 243d64a^ and is byte-identical to the 0.13.1 source cited in the seed report.
- The impacket_0_13_1 tag is not present in the MoSLoF/impacket fork — noted for the reproducibility section of the final report.

Read the patched CCache.getCredential (impacket/krb5/ccache.py:418–471) and the fix diff. The new branch `if c['server'].prettyPrint().count(b'/')

```
= 2 handles the 3-part SPN case ( service/host/domain@REALM`).
```

Read the shipped regression test at tests/misc/

test_ccache.py::test_ccache_getCredential_three_part_spn covering three cases: exact 3-part match, short-name FQDN match, and same-shortname-different-child-domain must-not-match.

Phase 2 — Fixture strategy

Rather than crafting raw ccache bytes, chose to build fixtures in-memory using the same API path that fromTGT / loadKirbiFile / test_ccache.py already exercise:

```
cred["server"] = Principal()
cred["server"].fromPrincipal(types.Principal(spn_string, type=NT_SRV_INST))
ccache.credentials.append(cred)
```

This avoids any live-AD dependency and produces fixtures whose construction logic is self-documenting and regenerable against future versions.

Phase 3 — Differential harness setup

Created two isolated trees for side-by-side runs:

```
$ git worktree add /tmp/.../prefix-tree 243d64a^
```

Installed the minimum dependency set (`pyasn1` , `six` , `pycryptodomex` , `pyOpenSSL` , `ldap3` , `flask`). Encountered the standard editable-install foot-gun: with `pip install -e .` present, `PYTHONPATH` was silently overridden by the meta-path finder; and even without an editable install, `sys.path[0]` (the cwd) shadows `PYTHONPATH` when running from inside the repo root. Verified controllable module resolution by running from a neutral cwd (`/tmp`) so that `PYTHONPATH` chose the tree:

```
$ (cd /tmp && PYTHONPATH=/tmp/.../prefix-tree python3 -c \
    "import impacket.krb5.ccache as m; print(m.__file__)"
/tmp/.../prefix-tree/impacket/krb5/ccache.py

$ (cd /tmp && PYTHONPATH=/home/user/impacket python3 -c \
    "import impacket.krb5.ccache as m; print(m.__file__)"
/home/user/impacket/impacket/krb5/ccache.py
```

Both trees now resolve independently.

Phase 4 — EC-01 through EC-05 harness

Wrote `scratchpad/harness.py` , a single script covering:

- EC-01: 3-part SPN direct search, poisoning-following-cred, exact match.
- EC-02: cross-realm referral TGT (`krbtgt/CHILD@PARENT`).
- EC-03: S4U2Self ST with no service type.
- EC-04: UTF-8 principal / realm.
- EC-05: port specifiers in 2-part and 3-part SPNs.
- Pathological shapes: 4-part SPN, no-slash search against a slash-bearing cache.

Ran the harness against both trees. Key results:

```
##### PREFIX (0.13.1-equiv, 2dae5c7) #####
[EC01-a direct-3part-search]      EXC IndexError: list index out of range
[EC01-b poison-following-cred]    EXC IndexError: list index out of range
[EC01-c exact-3part]             MATCH (fast-path)
[EC02-a rtgt-anyspn-nomatch]      NONE
[EC02-b rtgt-exact]              MATCH
[EC03-a nosvc-user]              NONE          (silent skip)
[EC03-b host-account]            MATCH
[EC04-a pretty-utf8]             b'caf\xe9...' (latin-1 mojibake)
[EC04-d cyrillic]                EXC UnicodeEncodeError
[EC05-a 2part-port-cache]        MATCH
[EC05-c 3part-port-cache]        EXC IndexError (2nd EC-01 trigger)
[EX-a 4part]                     EXC IndexError
[EX-b search-noslash-vs-2part]    EXC IndexError

##### MASTER (patched, 4c09897) #####
[EC01-a] MATCH [EC01-b] NONE [EC01-c] MATCH
[EC02*] correct on all
[EC03*] identical to baseline (silent-skip for user)
[EC04*] identical to baseline (mojibake + crash)
[EC05-c] MATCH [EX-a] MATCH
[EX-b] EXC IndexError          <-- residual, becomes EC-R
```

Calibration verdict: **PASS**. EC-01 crash reproduced (both direct and poisoning), absent on master. Also surfaced:

- EC-04 unchanged between versions — the `six.b()` latin-1 assumption underlies both mojibake and crash.
- EC-05 3-part-with-port confirmed as a second, independent trigger of the EC-01 crash class on baseline; fixed on master.
- **EX-b: a residual `IndexError` survives on master** for the class of no-slash search SPNs. Promoted to a first-class finding as EC-R.

Phase 5 — EC-04 layer analysis (`ec04.py`)

Wrote a focused script to determine *where* the UTF-8 failure originates, across layers: `six.b()`, `types.Principal()` construction, `fromPrincipal`, `prettyPrint`, and `getCredential`. Established:

- `six.b('café')` → `b'caf\xe9'` (latin-1), *not* the UTF-8 `b'caf\xc3\xa9'` a real ccache stores.
- A `Principal` loaded from *actual UTF-8 bytes* pretty-prints correctly (bytes pass through untouched); the break is on the string-construction and re-encoding paths.
- `six.b('MOCKBA')` → `UnicodeEncodeError` — hard crash.
- `getCredential("cifs/cephер@ДОМЕН.LOCAL")` crashes on its very first line, `b(server.upper())`.

Both versions identical; the EC-01 patch and the byte-normalisation commit (`8ea54fe`) do not touch this path.

Phase 6 — EC-06 expiry / flag ordering (`ec06.py`)

Confirmed by `grep` first that `endtime`, `renew_till`, and `tktfldgs` are parsed and stored but never referenced in `getCredential` selection. Constructed a fixture cache with:

- index 0: expired, non-forwardable ticket for `cifs/fs.lab.local`
- index 1: valid, forwardable ticket for the same SPN

Result on both versions:

```
Two creds for same SPN: [0]=expired/non-fwd, [1]=valid/forwardable
getCredential picked -> endtime=100000 flags=0x0
=> EXPIRED ticket selected (no expiry check)
```

Silent-wrong-result confirmed. Order-dependent; the "wrong" ticket wins on both versions.

Phase 7 — Cross-file assumption sweep

```
$ grep -rn "split(b'/' )\\[1\\]|split('/')\\[1\\]|split(b'@')\\[1\\]" impacket/krb5/
impacket/krb5/ccache.py:457-466
```

Pattern not duplicated elsewhere in `impacket/krb5/`. `kpasswd.py:363` calls `getCredential(principal, anySPN=False)` — bypasses the fragile loop entirely.

Reachability of the `krbtgt` path noted (`ccache.py:667`): every consumer that goes through `CCache.parseFile` triggers a `getCredential('krbtgt/DOMAIN@DOMAIN', anySPN=True)` call, which is why one bad cached ticket poisons the entire toolchain on the baseline.

Phase 8 — First deliverable: Kerberos ccache findings

Wrote `TM-022_Wave1_impacket_findings.md` in the required structured format:

- EC-XX header · spec basis · versions tested · files affected · outcome per version · same-assumption-elsewhere flag · fixture reference · operational notes.
- Executive summary matrix at the top with the EC-04/EC-06 promotion rationale spelled out.
- Recommendations section ordered by operational impact.

Committed and pushed:

```
$ git add TM-022_Wave1_impacket_findings.md tm022_wave1_harness/
$ git commit -m "TM-022 Wave 1: impacket krb5 ccache edge-case assessment"
$ git push -u origin claude/impacket-adversarial-wave-1-39koer
[new branch] claude/impacket-adversarial-wave-1-39koer
```

Reported completion to the operator.

Phase 9 — Ambiguous follow-up: "depSNORT assessment?"

Operator: "Did you perform the depSNORT assessment?"

Assessor: Reported honestly that "depSNORT" was not in the Wave 1 brief and was not a term recognised from the impacket code base or the briefing pack. Did not silently invent scope. Requested clarification.

Operator: clarified that depSNORT is their own tool at MoSLoF/depSNORT , to be used for the dependency assessment of impacket.

Phase 10 — Attaching the depSNORT repository

The GitHub MCP scope was limited to moslof/impacket only. Attempted add_repo MoSLoF/depSNORT — the call required an approval prompt in the operator's client. After brief back-and-forth on where the approval prompt was landing, the operator confirmed intent to attach:

```
add_repo(owner="MoSLoF", repo="depSNORT", access="read")
-> read available, clone at /workspace/moslof/depsnort
```

Cloned:

```
$ GIT_LFS_SKIP_SMUDGE=1 git clone --depth 1 \
  https://github.com/MoSLoF/depsnort /workspace/moslof/depsnort
Cloning into '/workspace/moslof/depsnort'...

$ git -C /workspace/moslof/depsnort rev-parse --short HEAD
5b02377
```

Read the full README. Identified depSNORT as a Go static supply-chain analyser, PyPI ecosystem in scope for impacket.

Phase 11 — Building and validating depSNORT

```
$ cd /workspace/moslof/depsnort
$ CGO_ENABLED=0 go build -o depsnort ./cmd/depsnort
$ ./depsnort version
depSNORT dev

$ ./depsnort checks
ID          AXIS              SEVERITY  GATE-CLASS    DESCRIPTION
VC-001      known-compromise  critical  block         package@version in known-malicious advisory
VC-002a     known-compromise  low       gate-eligible package declares an install lifecycle hook
... VC-002b/c/d/e/f, VC-003, VC-004, VC-005, VC-006, VC-007, VC-008 ...

$ ./depsnort sbom | python3 -c \
    "import json,sys; print('components:', \
        len(json.load(sys.stdin).get('components', [])))"
components: 0
```

Verified the dogfooding claim: SBOM's `components` array is empty (zero third-party deps). This will become a positive point in the tool assessment.

Noted: `version` reports `dev` for a source build, contrary to the README's confidence-check language that assumes a baked-in version string. Recorded as an observation for the tool-assessment writeup.

Phase 12 — depSNORT run #1: repo as-shipped

```
$ ./depsnort scan /home/user/impacket
depsnort: WARNING - coverage is incomplete: 10 unresolved
  dependenc(ies) across 1 root(s) ... This report is NOT an all-clear.
verdict.counts: {'block': 0, 'gate_eligible': 0, 'advisory': 0}
coverage.unresolved: 10
unresolved_names: [charset_normalizer, flask, ldap3, ldapdomaingroup,
  pyOpenSSL, pyasn1, pyasn1_modules, pycryptodomex, setuptools, six]
```

Finding DEP-01 emerges immediately: `impacket`'s `requirements.txt` is entirely unpinned, and depSNORT (per design decision D-01) refuses to resolve unpinned specifiers. All ten declared dependencies are reported as unresolved.

```
$ ./depsnort scan -fail-on-incomplete /home/user/impacket; echo $?
... WARNING - coverage is incomplete ...
3
```

Exit code 3 — the tool gates correctly on the coverage gap under `-fail-on-incomplete`.

Phase 13 — depSNORT run #2: pinned scan

Built a machine-generated pin set from an installed environment via `importlib.metadata`, then re-scanned:

```
$ cat depsnort-pinned-requirements.txt
setuptools==68.1.2
six==1.16.0
charset_normalizer==3.4.6
pyasn1==0.6.4
pyasn1_modules==0.4.2
pycryptodomex==3.23.0
pyOpenSSL==26.4.0
ldap3==2.9.1
ldapdomaindump==0.10.0
flask==3.1.3

$ ./depsnort scan ./depsnort-pinned-requirements-dir/
verdict.counts: {'block': 0, 'gate_eligible': 0, 'advisory': 1}
findings:
- VC-004 advisory medium | pkg:pypi/pyasn1@0.6.4
  title: 0.6.4 published after 492d of dormancy
ds: osv {queried:10, from_network:0, gaps:10} <-- 403 blocked
ds: pypi-registry {queried:10, from_cache:7, from_network:3, gaps:0}
```

Findings crystallise:

- **DEP-02 (advisory):** VC-004 dormancy on `pyasn1@0.6.4` (492 days). `pyasn1` is a load-bearing crypto dep for `impacket` — noted for prioritised recommendation.
- **DEP-03 (clean):** 10 install-hook nodes across the transitive tree, all with `capabilities: none`. `setuptools`'s `cmdclass.install` and `pycryptodomex`'s `cmdclass.build_ext` were correctly not misclassified as exfil-capable.
- **DEP-04 (environmental):** `api.osv.dev` denied by sandbox proxy (HTTP 403 CONNECT verified via `__agentproxy/status`).
- **DEP-05:** VC-007 dep-confusion not exercised (no internal scopes declared).

Emitted JSON, SARIF, DOT, and PDF outputs — all four rendered first-try.

Phase 14 — Second deliverable: depSNORT findings

Wrote `TM-022_Wave1_depSNORT_findings.md` in the same structured format as the `ccache` findings. Copied `depSNORT`'s own PDF into the repo as `TM-022_Wave1_depSNORT_report.pdf`.

Committed and pushed:

```
$ git add TM-022_Wave1_depSNORT_* tm022_wave1_harness/
$ git commit -m "TM-022 Wave 1: depSNORT dependency assessment of impacket"
$ git push
9bc39b4..b6b3215  claude/impacket-adversarial-wave-1-39koer
```

Sent the PDF report to the operator as a file attachment.

Phase 15 — Reporting brief

Operator: "Drop everything into a professional report for submission and also provide a separate assessment for the depSNORT tool." Mid-turn addition: "Oh, and a complete transcript of this session please."

Model switched to Opus 4.7 for the reporting phase.

Assessor: Installed `pandoc` (`apt install pandoc`, version 3.1.3), `reportlab`, and `weasyprint` to enable Markdown → HTML → PDF conversion.

Drafted three deliverables:

1. `reports/01_impacket_security_assessment.md` — consolidated professional report combining the ccache findings (§4, EC-01 through EC-R) and the supply-chain findings (§5, DEP-01 through DEP-05) with executive summary, recommendations, reproducibility appendix.
2. `reports/02_depSNORT_tool_assessment.md` — independent evaluation of depSNORT as a product: strengths, limitations, peer comparison, recommendations to the tool author.
3. `reports/03_session_transcript.md` — this document.

Converted each Markdown to PDF via `pandoc -t html5 | weasyprint`, committed all six files (three MD, three PDF), pushed to the working branch, and delivered the PDFs to the operator.

Phase 16 — Final state

Working branch on completion: `claude/impacket-adversarial-wave-1-39koer` (pushed to `origin`).

Commit history (assessment commits only, most recent first):

```
<hash>    TM-022 Wave 1: submission-ready reports and session transcript
b6b3215   TM-022 Wave 1: depSNORT dependency assessment of impacket
9bc39b4   TM-022 Wave 1: impacket krb5 ccache edge-case assessment
```

Total assessment deliverables (all on the working branch):

Path	Purpose
TM-022_Wave1_impacket_findings.md	Structured Kerberos ccache findings (source)
TM-022_Wave1_depSNORT_findings.md	Structured supply-chain findings (source)
TM-022_Wave1_depSNORT_report.pdf	depSNORT-generated PDF
tm022_wave1_harness/harness.py	EC-01/02/03/05 differential fixtures
tm022_wave1_harness/ec04.py	UTF-8 layer analysis
tm022_wave1_harness/ec06.py	Expiry / flag ordering test
tm022_wave1_harness/depsnort-scan-repo.json	Raw depSNORT JSON (unpinned)
tm022_wave1_harness/depsnort-scan-pinned.json	Raw depSNORT JSON (pinned)
tm022_wave1_harness/depsnort-scan-pinned.sarif	SARIF for CI ingestion
tm022_wave1_harness/depsnort-pinned-requirements.txt	Reproducible 10-of-10 pin set
reports/01_impacket_security_assessment.md / .pdf	Submission-ready consolidated report
reports/02_depSNORT_tool_assessment.md / .pdf	Tool evaluation
reports/03_session_transcript.md / .pdf	This transcript

Notes on transcript fidelity

- This transcript is a chronological reconstruction rather than a verbatim message log. All commands, exit codes, file paths, git commit hashes, numeric findings, and code excerpts are transcribed from the actual session outputs.
- Tool-call output blobs (JSON scans, full grep listings, complete harness prints) are elided where they would exceed a page; each phase cites the committed artefact where the full evidence lives.
- Two model changes are noted in-place: the assessment work through Phase 14 ran on Opus 4.8; the reporting phase (15 onward) ran on Opus 4.7 after an operator `/model` switch.
- One environmental limitation is disclosed explicitly rather than hidden: `api.osv.dev` was denied at the sandbox network gateway, affecting the OSV advisory tier of the depSNORT scan (DEP-04). All other tiers ran normally.

Session transcript, project iHBV-TM-022 (TM-022 Purple Validation Harness — Wave 1, impacket). Distribute only within the purple-team programme.